GENERATIVE AI

EXPT 1A. Text Generation using N-gram Language model

Code:

```
# ================================
# PREDICTIVE TEXT SYSTEM
# N-GRAM + HMM
# DATASET: BROWN CORPUS (URL BASED)
# ================================

# ----------- INSTALL & IMPORT -----------

import nltk
import random
from collections import defaultdict, Counter
from nltk.util import ngrams
from nltk.corpus import brown
from nltk.tag import hmm
from nltk import pos_tag

# Download required resources
nltk.download('brown')
nltk.download('universal_tagset')
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('punkt_tab')

# ----------- LOAD DATA -----------

print("Loading Brown Corpus...")

words = brown.words()
sentences = brown.sents()
tagged_sentences = brown.tagged_sents(tagset='universal')

print("Total words:", len(words))
print("Total sentences:", len(sentences))
```

```python
# ----------- PREPROCESSING -----------

tokens = [word.lower() for word in words if word.isalpha()]
print("Total processed tokens:", len(tokens))
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Package universal_tagset is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Loading Brown Corpus...
Total words: 1161192
Total sentences: 57340
Total processed tokens: 981716
```

```python
# ==========================================================
#                 N-GRAM MODEL
# ==========================================================

# Function to build N-gram model
def build_ngram_model(tokens, n):
    model = defaultdict(Counter)

    for gram in ngrams(tokens, n):
        context = tuple(gram[:-1])
        prediction = gram[-1]
        model[context][prediction] += 1

    return model
```

```python
# Function to predict next words
def predict_next_ngram(text, model, k=5):

    context_words = tuple(text.lower().split())

    if not model:
        return {}

    first_key = next(iter(model.keys()))
    n_minus_1 = len(first_key)

    context = context_words[-n_minus_1:]

    if context in model:
        return dict(model[context].most_common(k))
    else:
        return {}


print("\nBuilding N-Gram Models...")
bigram_model = build_ngram_model(tokens, 2)
trigram_model = build_ngram_model(tokens, 3)
```

```
Building N-Gram Models...
```

```python
# ----------- N-GRAM TESTING WITH POS TAGS -----------

print("\n=========== N-GRAM MODEL TESTING WITH POS TAGS ==========")

context_bigram = "the"
context_trigram = "in the"

# ----- BIGRAM -----
print("\nInput (Bigram):", context_bigram)
```

```python
input_tokens = context_bigram.split()
print("Input POS Tags:", pos_tag(input_tokens))

bigram_predictions = predict_next_ngram(context_bigram, bigram_model)
pred_words = list(bigram_predictions.keys())

print("Predicted Words:", pred_words)
print("Output POS Tags:", pos_tag(pred_words))


# ----- TRIGRAM -----
print("\nInput (Trigram):", context_trigram)

input_tokens = context_trigram.split()
print("Input POS Tags:", pos_tag(input_tokens))

trigram_predictions = predict_next_ngram(context_trigram, trigram_model)
pred_words = list(trigram_predictions.keys())

print("Predicted Words:", pred_words)
print("Output POS Tags:", pos_tag(pred_words))
```

```
=========== N-GRAM MODEL TESTING WITH POS TAGS ==========

Input (Bigram): the
Input POS Tags: [('the', 'DT')]
Predicted Words: ['first', 'same', 'most', 'other', 'new']
Output POS Tags: [('first', 'RB'), ('same', 'JJ'), ('most', 'RBS'), ('other', 'JJ'), ('new', 'JJ')]

Input (Trigram): in the
Input POS Tags: [('in', 'IN'), ('the', 'DT')]
Predicted Words: ['first', 'world', 'united', 'same', 'past']
Output POS Tags: [('first', 'JJ'), ('world', 'NN'), ('united', 'JJ'), ('same', 'JJ'), ('past', 'NN')]
```

EXPT 1B. Hidden Markov Model (HMM) based Predictive Text System

Code:

```
# ============================================================
#                 HMM MODEL
# ============================================================

print("\nTraining HMM Tagger...")

train_size = int(len(tagged_sentences) * 0.9)
training_data = tagged_sentences[:train_size]

hmm_tagger = hmm.HiddenMarkovModelTagger.train(training_data)

print("HMM Tagger training complete.")

# Build POS → word mapping
pos_word_map = defaultdict(list)

for sent in training_data:
    for word, tag in sent:
        pos_word_map[tag].append(word)


# Function to predict next words using HMM
def predict_next_hmm(sentence):

    tokens = nltk.word_tokenize(sentence.lower())

    tagged_sentence = hmm_tagger.tag(tokens)
    last_tag = tagged_sentence[-1][1]

    if last_tag in pos_word_map:
        word_freq = Counter(pos_word_map[last_tag])
        return [w for w, _ in word_freq.most_common(5)]
    else:
        return ["No prediction"]
```

```
Training HMM Tagger...
HMM Tagger training complete.
```

```python
# ----------- HMM TESTING WITH POS TAGS -----------

print("\n=========== HMM MODEL TESTING WITH POS TAGS ==========")

sent1 = "The quick brown fox"
sent2 = "She is running"


# ----- EXAMPLE 1 -----
print("\nInput Sentence 1:", sent1)

input_tokens = nltk.word_tokenize(sent1)
print("Input POS Tags:", pos_tag(input_tokens))

pred_words = predict_next_hmm(sent1)

print("Predicted Words:", pred_words)
print("Output POS Tags:", pos_tag(pred_words))


# ----- EXAMPLE 2 -----
print("\nInput Sentence 2:", sent2)

input_tokens = nltk.word_tokenize(sent2)
print("Input POS Tags:", pos_tag(input_tokens))

pred_words = predict_next_hmm(sent2)

print("Predicted Words:", pred_words)
print("Output POS Tags:", pos_tag(pred_words))


print("\n===== PROGRAM COMPLETED SUCCESSFULLY =====")
```

```
=========== HMM MODEL TESTING WITH POS TAGS ==========

Input Sentence 1: The quick brown fox
Input POS Tags: [('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN')]
Predicted Words: ['time', 'man', 'Af', 'years', 'Mr.']
Output POS Tags: [('time', 'NN'), ('man', 'NN'), ('Af', 'NNP'), ('years', 'NNS'), ('Mr.', 'NNP')]

Input Sentence 2: She is running
Input POS Tags: [('She', 'PRP'), ('is', 'VBZ'), ('running', 'VBG')]
Predicted Words: ['is', 'was', 'be', 'had', 'are']
Output POS Tags: [('is', 'VBZ'), ('was', 'VBD'), ('be', 'VB'), ('had', 'VBN'), ('are', 'VBP')]

===== PROGRAM COMPLETED SUCCESSFULLY =====
```

GOOGLE COLLAB LINK : GenAI EX-1.ipynb